

RELATÓRIO TÉCNICO

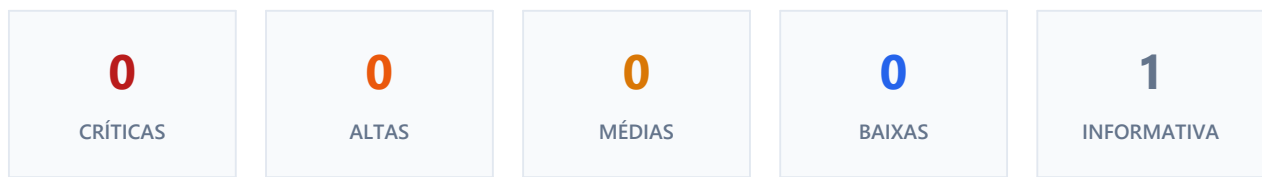
Relatório de Auditoria de Segurança — FDT Sales Manager

Auditoria orientada às cinco categorias solicitadas, adaptadas à arquitetura desktop Python/SQLite.

DATA	03/09/2026
ESCOPO	Código atual, histórico Git, SQLite, GUI, relatório HTML, scripts e configuração PyInstaller.
STACK	Python + CustomTkinter + sqlite3 + Jinja2; sem API, ORM, autenticação ou frontend web.
RESULTADO	0 vulnerabilidades confirmadas · 0 configurações perigosas · 1 melhoria defensiva informativa

Nota metodológica: banco sem tranca foi mapeado para o isolamento real pelo usuário/processo do Windows; permissão no navegador e IDOR foram avaliados pela ausência/presença de rotas, identidades e gates; segredos foram verificados no estado atual e histórico; XSS foi validado estaticamente e com payloads em base temporária.

1. Resumo executivo



Não foi demonstrado caminho explorável nas cinco categorias. A única observação acionável é preventiva: adicionar Content-Security-Policy e testes permanentes para todos os campos persistidos do relatório HTML.

2. Stack e adaptação das categorias

Camada	Deteção	Implicação para a auditoria
Aplicação	Python desktop / CustomTkinter	Sem servidor ou navegador como frontend
Dados	SQLite via sqlite3; sem ORM	107 chamadas SQL revisadas diretamente
Auth	Nenhuma identidade/sessão/papel	Trust boundary: conta/processo do Windows
HTML	Jinja2 + SVG em data URI	Autoescape e sinks manuais auditados
Deploy	PyInstaller onedir	Sem Docker, CI, Helm ou Terraform

- Banco sem tranca: verificação do isolamento por conta do Windows e diretório LocalAppData; não existe tenant de aplicação.
- Permissão no navegador: busca de gates por papel e cruzamento de todas as ações da GUI com seus serviços.
- IDOR: enumeração AST de todos os handlers (0 HTTP) e revisão dos 37 métodos públicos de negócio.
- Chaves expostas: workspace, scripts, documentação, commits e bundle PyInstaller final.
- XSS: sinks HTML/JS, template, SVG, URLs, autoescape e teste dinâmico com dados persistidos.

3. Pontos fortes e pontos fracos

Pontos fortes

- Jinja2 com autoescape para HTML/XML (`relatorio\_html\_service.py:268-271`).
- Escape manual antes de inserir texto nos SVGs (`relatorio\_html\_service.py:101-115, 165-169`).
- Valores SQL externos usam placeholders; as únicas duas f-strings SQL contêm identificadores internos fixos.
- Transações com rollback em pedido, conversão de lead e importação CSV.
- Foreign keys e constraints de integridade ativadas no SQLite (`database.py:15-18, 295-428`).
- No build, dados graváveis ficam no LocalAppData por usuário (`paths.py:32-38`).
- Banco `.db` ignorado e ausente do histórico versionado.

Pontos fracos / limitações

- Ausência de CSP e de testes versionados para todos os campos persistidos exibidos no HTML (I-001, informativa).
- A arquitetura local precisará de novo threat model antes de qualquer API, sincronização ou base compartilhada.
- A suíte geral chegou a 59% antes do timeout; a suíte específica de relatório passou 5/5.

4. Resultado detalhado por categoria

Categoria	Resultado	Evidência principal
1. Banco sem tranca	Não aplicável como vulnerabilidade	Base local por usuário; sem tenant/auth (`paths.py:27-38`)
2. Permissão no navegador	Não aplicável	GUI desktop; zero gates de papel; matriz completa auditada
3. IDOR	Não aplicável	0 handlers HTTP; 37 métodos de negócio inventariados
4. Chaves expostas	Nenhum achado	0 no workspace/histórico; bundle final varrido
5. XSS	0 vulnerabilidades; 1 melhoria informativa	Autoescape + escape SVG + teste dinâmico aprovado

5. Tabela de achados

Severidade	Arquivo:linha	Descrição
INFORMATIVA	src/templates/relatorio_comercial.html:3-8 tests/test_relatorio_html_service.py:92-103	Reforçar defesa em profundidade e testes de regressão XSS do relatório HTML Classificação: melhoria defensiva; não é vulnerabilidade atual.

I-001 - Reforçar defesa em profundidade e testes XSS

O template não declara Content-Security-Policy e o teste versionado cobre apenas o título. A auditoria dinâmica confirmou que cliente, produto, referência e SVG estão escapados na versão atual; portanto, não há exploração atual demonstrada.

```
# src/services/relatorio_html_service.py:268-271
ambiente = Environment(
    loader=FileSystemLoader(TEMPLATES_DIR),
    autoescape=select_autoescape(("html", "xml")),
)

# tests/test_relatorio_html_service.py:92-103
titulo="<script>alert('x')</script>"
assert "<script>alert" not in conteudo
assert "&lt;script&gt;" in conteudo
```

Condição de explorabilidade: exigiria regressão futura no escape ou novo sink HTML inseguro, seguida da abertura do relatório com dado malicioso.

6. Recomendações priorizadas

Prioridade	Ação	Motivo
P1	Nenhuma correção urgente	0 vulnerabilidades confirmadas
P2	Automatizar secret scan no histórico e bundle em CI	Transformar a verificação manual em controle contínuo
P2	Revisar threat model antes de API/base compartilhada	Tenant/auth/IDOR hoje não se aplicam
P3	Adicionar CSP e testes XSS amplos	I-001, defesa em profundidade

Cobertura e limitações

- A suíte completa coletou 156 testes e avançou até 59% antes do timeout de 120 segundos. Os 5 testes do relatório passaram em execução isolada.
- Nenhum serviço externo, produção ou banco real foi acessado; o teste dinâmico XSS usou SQLite temporário.

7. ISSUES PARA O GITHUB

--- ISSUE 1 ---

Título

[Segurança] Reforçar defesa contra regressões XSS no relatório HTML

Labels sugeridas

`security`, `informativa`

Descrição do problema e explorabilidade

O relatório está protegido atualmente por autoescape do Jinja2 e escape manual dos SVGs; não há XSS explorável confirmado. Como defesa em profundidade, falta CSP e o teste versionado cobre apenas o título. Uma regressão futura com `safe`, `Markup` ou HTML manual poderia executar conteúdo quando o operador abrisse um relatório malicioso.

Evidência

```
`src/templates/relatorio_comercial.html:3-8`

```html
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>{{ titulo }}</title>
 <link rel="icon" type="image/png" href="{{ favicon_data_uri }}">
 <style>
 ...
```

Não há meta CSP no cabeçalho.

```
`tests/test_relatorio_html_service.py:92-103`

```python
def test_relatorio_escapa_titulo_html(...):
    titulo="<script>alert('x')</script>"
    assert "<script>alert" not in conteudo
    assert "&lt;script&gt;" in conteudo
    ...
```

O teste cobre o título, mas não os demais campos persistidos nem o SVG decodificado.

Impacto

Defesa preventiva contra uma futura regressão de XSS em relatórios locais. Não há vulnerabilidade explorável confirmada na versão atual.

Sugestão de correção

1. Adicionar CSP compatível com o documento autônomo: `default-src 'none'; img-src data;; style-src 'unsafe-inline'; base-uri 'none'; form-action 'none'`.
2. Criar testes parametrizados para título, cliente, produto, categoria, referência externa e nome de arquivo.
3. Decodificar os SVGs base64 e verificar ausência do payload bruto e presença da versão escapada.
4. Manter proibidos `|safe`, `Markup`, `mark\_safe`, HTML manual com dados e URLs de usuário sem validação.

Critérios de aceite

- [] O HTML inclui CSP documentada e compatível com os recursos locais.
- [] Gráficos e estilos continuam funcionando offline.
- [] Testes cobrem título, cliente, produto, categoria, referência e nome de arquivo.
- [] Testes decodificam os SVGs e verificam o escape.
- [] Nenhum payload bruto aparece no HTML ou SVG.
- [] `tests/test\_relatorio\_html\_service.py` permanece verde.

--- FIM ISSUE 1 ---

8. Evidências de validação

Resultados automatizados

Verificação	Resultado
Enumeração de rotas	0 handlers HTTP
Métodos públicos de negócio	37 revisados + 1 serializador DTO
Chamadas SQL	107; 105 literais e 2 f-strings internas de migração
Segredos no histórico	0 correspondências
Teste dinâmico XSS	10/10 assertivas de escape aprovadas
Testes de relatório	5/5 aprovados em 8,84 s
Bundle	1.907 arquivos / 115.981.209 bytes; 0 segredos confirmados

Arquivos complementares

- docs/security-audit/relatorio-auditoria-seguranca.md
- docs/security-audit/achados.json
- docs/security-audit/inventario-rotas.md
- docs/security-audit/metodologia.md
- docs/security-audit/gerar_relatorio_pdf.py

Conclusão: nenhuma das cinco classes apresentou vulnerabilidade explorável na arquitetura e no código auditados. A observação I-001 é explicitamente preventiva.